

Trees and Graphs: Traverse with a Purpose

A traversal is a framework for exploring a state space while preserving the information needed by the problem. Decide what a node and edge represent, whether the graph is directed or weighted, and whether you need reachability, shortest distance, ordering, or an aggregate over subtrees.

For: Learners who know BFS and DFS names but need to choose traversal state, visitation timing, and graph modeling correctly for unfamiliar problems.

Suggested pace: 3 weeks of traversal and modeling practice | Recursion and queues understood

What you will be able to do

- Model graph state and choose adjacency representations from the workload.
- Use BFS for equal-cost shortest paths and DFS for appropriate structural reasoning.
- Handle cycles, disconnected components, and recursion depth deliberately.
- Explain topological ordering and why weighted paths need additional assumptions.

Build the right mental model

Model before traversing

A node might be a cell, a word, a task, or a compound state such as position plus remaining budget. If future choices depend on information not in the node identity, the visited set can incorrectly merge different states.

Visited timing controls work

For ordinary BFS, marking on enqueue prevents duplicate queue entries and preserves a clean layer interpretation. DFS cycle detection on a directed graph needs active-path state, not just one global visited bit. Undirected traversal must account for the parent edge.

Traversal order carries meaning

BFS discovers minimum edge count in an unweighted graph. Postorder can compute child results before a parent. A topological order respects dependencies only when the directed graph is acyclic. Choose the order because the proof needs it.

Study with evidence.

Draw a small graph and annotate the queue or stack at each step. Include a cycle and a disconnected component. Implement both recursive and iterative versions where useful, and account for call-stack depth rather than treating recursion as free memory.

Week 1: Tree state

Practice preorder, inorder, postorder, level order, height, and path constraints. Define what each recursive call returns versus what is accumulated globally. Test empty and skewed trees and compare node identity with stored values.

Deliverable and check: A traversal trace and a subtree computation with a clear return-value invariant.

Week 2: Graph exploration

Implement adjacency lists, grid neighbors, components, BFS distances, and cycle detection. Decide when to mark visited and whether mutation of the input is allowed. Compare $O(V+E)$ traversal with dense matrix costs.

Deliverable and check: Tests for cycles, duplicate edges where allowed, isolated nodes, and unreachable destinations.

Week 3, first half: Dependencies and weights

Study indegrees and topological sorting, then compare BFS with Dijkstra under nonnegative weights. Explain why negative edges violate Dijkstra's settled-distance argument and why a stop budget may belong in the state.

Deliverable and check: A dependency-order example and a shortest-path decision table tied to edge assumptions.

Week 3, second half: Unseen modeling

Solve a transformation, scheduling, or constrained-path problem without a visible graph label. State the node identity, transitions, objective, and visited contract before choosing a traversal.

Deliverable and check: A new problem modeled correctly and checked with a counterexample to an oversimplified state.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Schedule tasks and detect a dependency cycle

Tasks A and B must finish before C; C must finish before D. You need a valid execution order or a clear report that the dependency graph contains a cycle.

A worked approach

1. Create edges $A \rightarrow C$, $B \rightarrow C$, and $C \rightarrow D$, with indegrees zero, zero, two, and one. Enqueue every zero-indegree task, not just an arbitrary starting node, because the graph may have several independent sources.
2. Remove A and B in either order, decrementing C's indegree for each completed dependency. C becomes ready only after both decrements. Removing C then makes D ready. One valid result is A, B, C, D.
3. Add edge $D \rightarrow B$. After processing A, the remaining B-C-D cycle never reaches zero indegree. If the number processed is less than the number of tasks, report a cycle rather than returning a misleading partial schedule.

Expected result

The indegree invariant counts unresolved prerequisites. The algorithm takes $O(V+E)$ time with adjacency lists and detects that no complete topological order exists when a cycle remains.

Verify the result

- Are all initial zero-indegree nodes included?
- Does every edge decrement its destination exactly once?
- Is a partial result rejected when not all tasks were processed?

Extend the exercise

Add task durations and ask for the earliest completion time under unlimited workers. Propagate the maximum predecessor finish time along the topological order; do not confuse this with resource-limited scheduling.

Make the learning stick.

Why is BFS not always the right shortest-path algorithm?

It minimizes edge count when each edge has equal cost. Different nonnegative weights need an algorithm such as Dijkstra; negative weights require other reasoning. The graph's weight contract determines the choice.

Can I use one visited bit for every cycle problem?

No. Directed-cycle detection often needs unvisited, active, and finished states, or an indegree-based method. Seeing a previously finished node is not the same as finding a back edge to the active path.

What if recursive DFS overflows the stack?

Use an explicit stack or another traversal when depth can be large. State the input bound and memory model; a correct recursive idea still needs an implementation that handles the permitted depth.

Avoid these traps

- Using an incomplete node identity for a constrained path problem.
- Assuming a visited node always implies a directed cycle.
- Ignoring disconnected components or recursion-depth limits.

Ready for the next step?

- Node, edge, and visited-state meanings are explicit.
- Traversal order is justified by the objective.
- Cycles and unreachable states are handled correctly.
- Complexity is expressed in the relevant vertices, edges, or grid cells.

Reliable next reads

- [MIT algorithms course](#)
- [CP-Algorithms: graph algorithms](#)
- [Python deque](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.